



Trabalhando com Listas em Python

Manipule coleções de dados de forma eficiente

Objetivos da Aula

- ✓ Compreender o conceito de listas e sua importância na programação
- ✓ Aprender a criar e acessar elementos de listas em Python
- ✓ Dominar o uso dos métodos `append()`, `insert()` e `remove()`
- ✓ Entender e aplicar o fatiamento (slicing) de listas
- ✓ Praticar a manipulação de listas através de exercícios práticos
- ✓ Desenvolver habilidades para resolver problemas usando listas



O que são Listas?

Definição

Uma **lista** é uma coleção ordenada e mutável de elementos que podem ser de diferentes tipos de dados. Em Python, as listas são representadas por colchetes `[]`.

Características Principais

- **Ordenadas**: Os elementos têm uma posição definida
- **Mutáveis**: Podem ser modificadas após a criação
- **Indexadas**: Acessadas por índices (começando em 0)
- **Heterogêneas**: Podem conter diferentes tipos de dados
- **Duplicadas**: Permitem elementos repetidos



Exemplo de lista:

```
frutas = ["maçã", "banana", "laranja", "uva"]
numeros = [1, 2, 3, 4, 5]
misturada = ["texto", 42, 3.14, True]
```



Criação e Acesso a Listas

Criando Listas

```
# Lista vazia
lista_vazia = []

# Lista com elementos
frutas = ["maçã", "banana", "laranja"]

# Lista com diferentes tipos
misturada = [1, "texto", 3.14, True]

# Usando list()
numeros = list(range(1, 6)) # [1, 2, 3, 4, 5]
```

Acessando Elementos

```
frutas = ["maçã", "banana", "laranja", "uva"]

# Acesso por índice (começa em 0)
```



```
print(frutas[0]) # "maçã"
print(frutas[2]) # "laranja"

# Índice negativo (do final para o início)
print(frutas[-1]) # "uva"
print(frutas[-2]) # "laranja"
```

Importante: O primeiro elemento tem índice 0, o segundo tem índice 1, e assim por diante. Índices negativos contam do final para o início.



Método append()

O que faz o append()?

O método **append()** adiciona um elemento ao **final** da lista. É uma das formas mais comuns de adicionar elementos a uma lista existente.

Sintaxe

```
lista.append(elemento)
```

Exemplos Práticos

```
# Criando uma lista vazia
frutas = []

# Adicionando elementos
frutas.append("maçã")
frutas.append("banana")
frutas.append("laranja")
```



< >

5 / 19

```
print(frutas) # ["maçã", "banana", "laranja"]
```

```
# Adicionando um número
```

```
numeros = [1, 2, 3]
```

```
numeros.append(4)
```

```
print(numeros) # [1, 2, 3, 4]
```

Características: O `append()` sempre adiciona o elemento no final da lista e modifica a lista original (não retorna uma nova lista).



Método insert()

O que faz o insert()?

O método **insert()** adiciona um elemento em uma **posição específica** da lista, deslocando os elementos existentes para a direita.

Sintaxe

```
lista.insert(índice, elemento)
```

Exemplos Práticos

```
frutas = ["maçã", "banana", "laranja"]
```

```
# Inserindo no início (índice 0)
frutas.insert(0, "uva")
print(frutas) # ["uva", "maçã", "banana",
               "laranja"]
```

```
# Inserindo no meio (índice 2)
```



< >

```
frutas.insert(2, "morango")  
print(frutas) # ["uva", "maçã", "morango",  
"banana", "laranja"]  
  
# Inserindo no final  
frutas.insert(len(frutas), "abacaxi")  
print(frutas) # ["uva", "maçã", "morango",  
"banana", "laranja", "abacaxi"]
```

Dica: Use insert(0, elemento) para adicionar no início e insert(len(lista), elemento) para adicionar no final (equivalente ao append()).



Método remove()

O que faz o remove()?

O método **remove()** remove a **primeira ocorrência** de um elemento específico da lista. Se o elemento não existir, gera um erro.

Sintaxe

```
lista.remove(elemento)
```

Exemplos Práticos

```
frutas = ["maçã", "banana", "laranja", "banana"]
```

```
# Removendo "banana" (primeira ocorrência)
```

```
frutas.remove("banana")
```

```
print(frutas) # ["maçã", "laranja", "banana"]
```

```
# Removendo "laranja"
```

```
frutas.remove("laranja")  
print(frutas) # ["maçã", "banana"]  
  
# Tentando remover elemento inexistente  
# frutas.remove("uva") # Erro: ValueError
```



Verificando antes de remover

```
frutas = ["maçã", "banana", "laranja"]  
  
if "uva" in frutas:  
    frutas.remove("uva")  
else:  
    print("Elemento não encontrado!")
```

Atenção: O remove() remove apenas a primeira ocorrência. Para remover todas as ocorrências, use um laço ou list comprehension.



Fatiamento de Listas

O que é Fatiamento?

O **fatiamento (slicing)** permite extrair uma parte específica de uma lista usando índices de início e fim. É uma técnica muito poderosa para trabalhar com listas.

Sintaxe

```
lista[início:fim:passo]
```

Exemplos Práticos

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Elementos do índice 2 ao 5 (exclusivo)  
print(numeros[2:6]) # [2, 3, 4, 5]
```

```
# Do início até o índice 4  
print(numeros[:5]) # [0, 1, 2, 3, 4]
```



```
# Do índice 5 até o final
print(numeros[5:]) # [5, 6, 7, 8, 9]

# Com passo de 2
print(numeros[::2]) # [0, 2, 4, 6, 8]

# Lista invertida
print(numeros[::-1]) # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Dica: O fatiamento não modifica a lista original, ele cria uma nova lista com os elementos selecionados.



Exercícios Práticos - Parte 1

1. Lista de Compras

Crie uma lista vazia chamada "compras" e adicione 5 itens usando o método `append()`.

```
compras = []  
# Adicione: "pão", "leite", "ovos", "açúcar", "café"
```

2. Inserir no Meio

Dada a lista `[1, 2, 4, 5]`, use `insert()` para adicionar o número 3 na posição correta.

```
numeros = [1, 2, 4, 5]  
# Insira 3 na posição 2
```

3. Remover Elemento

Crie uma lista com frutas duplicadas e remova apenas a primeira ocorrência de "banana".

Pratique no [Google Colab](#)
para fixar os conceitos!

< >

9 / 19

```
frutas = ["maçã", "banana", "laranja", "banana", "uva"]  
# Remova a primeira "banana"
```

4. Fatiamento Básico

Dada a lista [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], extraia os elementos do índice 3 ao 7.

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
# Use fatiamento para extrair [3, 4, 5, 6, 7]
```



Exercícios Práticos - Parte 2

5. Lista de Notas

Crie uma lista com 5 notas, calcule a média e adicione a média na lista usando `append()`.

```
notas = [8.5, 7.0, 9.2, 6.8, 8.0]
# Calcule a média e adicione à lista
```

6. Inserir no Início

Dada uma lista de nomes, insira "Ana" no início da lista.

```
nomes = ["João", "Maria", "Pedro"]
# Insira "Ana" no início
```

7. Fatiamento Avançado

Dada a lista [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], extraia apenas os números pares usando fatiamento.

Continue praticando para dominar as listas!

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
# Use [::2] para pegar de 2 em 2
```

8. Lista Invertida

Crie uma lista com 5 elementos e exiba ela invertida usando fatiamento.

```
lista = ["A", "B", "C", "D", "E"]  
# Use [::-1] para inverter
```



Resumo e Próximos Passos

O que aprendemos hoje?

Criação de Listas

[] ou list()

```
lista = [1, 2, 3]
```

Acesso por Índice

lista[índice]

```
lista[0] ou lista[-1]
```

append()

Adiciona no final

```
lista.append(item)
```

insert()

Insere em posição específica

```
lista.insert(pos, item)
```

remove()

Remove primeira ocorrência

Fatiamento

Extraí parte da lista



< >

```
lista.remove(item)
```

```
lista[início:fim]
```

Próximos Passos

- Pratique os exercícios no [Google Colab](#)
- Explore outros métodos de listas (sort, reverse, count, etc.)
- Aprenda sobre [tuplas](#) e [dicionários](#)
- Use listas em projetos práticos
- Continue estudando estruturas de dados em Python



Método pop()

O que faz o pop()?

O método **pop()** remove e **retorna** o elemento de uma posição específica da lista. Se nenhum índice for fornecido, remove o último elemento.

Sintaxe

```
elemento = lista.pop(índice) # Remove elemento do índice  
elemento = lista.pop() # Remove último elemento
```

Exemplos Práticos

```
frutas = ["maçã", "banana", "laranja", "uva"]  
  
# Removendo o último elemento  
ultima = frutas.pop()  
print(f"Removido: {ultima}") # "Removido: uva"  
print(frutas) # ["maçã", "banana", "laranja"]
```



< >

```
# Removendo elemento específico  
segunda = frutas.pop(1)  
print(f"Removido: {segunda}") # "Removido: banana"  
print(frutas) # ["maçã", "laranja"]
```

Vantagem: O pop() retorna o elemento removido, permitindo que você o use em outras operações.



Métodos sort() e reverse()

Método sort()

O método **sort()** ordena os elementos da lista **in-place** (modifica a lista original).

Método reverse()

O método **reverse()** inverte a ordem dos elementos da lista **in-place**.

Exemplos Práticos

```
# Ordenação
numeros = [3, 1, 4, 1, 5, 9, 2]
print("Original:", numeros)

numeros.sort()
print("Ordenado:", numeros) # [1, 1, 2, 3, 4, 5, 9]
```

```
# Ordenação decrescente
```



< >

```
numeros.sort(reverse=True)
print("Decrescente:", numeros) # [9, 5, 4, 3, 2, 1,
1]

# Inversão
frutas = ["maçã", "banana", "laranja"]
frutas.reverse()
print("Invertido:", frutas) # ["laranja", "banana",
"maçã"]
```

Importante: Ambos os métodos modificam a lista original e não retornam uma nova lista.



Métodos count() e index()

Método count()

O método **count()** retorna o **número de ocorrências** de um elemento na lista.

Método index()

O método **index()** retorna o **índice** da primeira ocorrência de um elemento na lista.

Exemplos Práticos

```
numeros = [1, 2, 3, 2, 4, 2, 5]
```

```
# Contando ocorrências
```

```
print(numeros.count(2)) # 3
```

```
print(numeros.count(7)) # 0
```

```
# Encontrando índices
```

```
print(numeros.index(2)) # 1 (primeira ocorrência)
```



< >

```
print(numeros.index(4)) # 4

# Busca a partir de uma posição
print(numeros.index(2, 2)) # 3 (busca a partir do
índice 2)

# Tratamento de erro
if 7 in numeros:
    print(numeros.index(7))
else:
    print("Elemento não encontrado!")
```

Atenção: O `index()` gera erro se o elemento não for encontrado. Use `'in'` para verificar antes.



Método extend()

O que faz o extend()?

O método **extend()** adiciona **todos os elementos** de uma sequência (lista, tupla, string) ao final da lista atual.

Sintaxe

```
lista.extend(sequência)
```

Exemplos Práticos

```
# Lista original  
frutas = ["maçã", "banana"]  
print("Original:", frutas)
```

```
# Estendendo com outra lista  
novas_frutas = ["laranja", "uva", "morango"]  
frutas.extend(novas_frutas)  
print("Após extend:", frutas)
```

```
# ["maçã", "banana", "laranja", "uva", "morango"]
```

```
# Estendendo com string
```

```
frutas.extend("kiwi")
```

```
print("Com string:", frutas)
```

```
# ["maçã", "banana", "laranja", "uva", "morango",  
"k", "i", "w", "i"]
```

```
# Estendendo com tupla
```

```
numeros = [1, 2, 3]
```

```
numeros.extend((4, 5, 6))
```

```
print("Com tupla:", numeros) # [1, 2, 3, 4, 5, 6]
```



Diferença: extend() adiciona elementos individuais, enquanto append() adiciona a sequência como um único elemento.



Métodos clear() e copy()

Método clear()

O método **clear()** remove **todos os elementos** da lista, deixando-a vazia.

Método copy()

O método **copy()** cria uma **cópia superficial** da lista (nova lista com os mesmos elementos).

Exemplos Práticos

```
# Método clear()
lista = [1, 2, 3, 4, 5]
print("Antes:", lista)

lista.clear()
print("Após clear:", lista) # []
print("Tamanho:", len(lista)) # 0
```

```
# Método copy()
original = ["a", "b", "c"]
copia = original.copy()

print("Original:", original)
print("Cópia:", copia)
```

```
# Modificando a cópia
copia.append("d")
print("Original:", original) # ["a", "b", "c"]
print("Cópia:", copia) # ["a", "b", "c", "d"]
```



Importante: copy() cria uma nova lista independente. Mudanças na cópia não afetam a original.



Funções len() e in

Função len()

A função **len()** retorna o **número de elementos** em uma lista.

Operador in

O operador **in** verifica se um elemento **está presente** na lista, retornando True ou False.

Exemplos Práticos

```
frutas = ["maçã", "banana", "laranja", "uva"]

# Usando len()
print("Tamanho da lista:", len(frutas)) # 4
print("Primeiro elemento:", frutas[0])
print("Último elemento:", frutas[len(frutas)-1])

# Usando in
print("maçã" in frutas) # True
```

```
print("abacaxi" in frutas) # False
```

```
# Aplicações práticas
```

```
if "banana" in frutas:  
    print("Banana encontrada!")
```

```
if len(frutas) > 0:  
    print("Lista não está vazia")
```

```
# Verificando antes de remover
```

```
if "laranja" in frutas:  
    frutas.remove("laranja")
```



Dica: Sempre use 'in' antes de operações que podem gerar erro, como remove() ou index().



Exercícios com Métodos Avançados

1. Gerenciamento de Lista

Crie uma lista com 5 números, ordene-a, remova o último elemento e conte quantas vezes o número 5 aparece.

```
numeros = [3, 5, 1, 5, 2]
# Ordene, remova último e conte 5s
```

2. Busca e Remoção

Dada uma lista de nomes, verifique se "Ana" existe, encontre sua posição e remova-a usando pop().

```
nomes = ["João", "Maria", "Ana", "Pedro"]
# Verifique, encontre posição e remova
```

3. Combinação de Listas

Crie duas listas, combine-as usando extend(), inverta a ordem e faça uma cópia da lista resultante.

Pratique todos os métodos aprendidos!

< >

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
# Combine, inverta e copie
```

4. Análise Completa

Crie uma lista com números duplicados, ordene-a, remova duplicatas e mostre o tamanho final.

```
numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5]
# Ordene, remova duplicatas, mostre tamanho
```



Resumo Completo de Métodos

Métodos de Adição

append()

Adiciona elemento no final

```
lista.append(item)
```

insert()

Insere em posição específica

```
lista.insert(pos, item)
```

extend()

Adiciona elementos de sequência

```
lista.extend(seq)
```

Métodos de Remoção

remove()

Remove primeira ocorrência

pop()

Remove e retorna elemento

```
lista.remove(item)
```

```
lista.pop(índice)
```

clear()

Remove todos os elementos

```
lista.clear()
```



Métodos de Ordenação e Busca

sort()

Ordena a lista

```
lista.sort()
```

reverse()

Inverte a ordem

```
lista.reverse()
```

count()

Conta ocorrências

```
lista.count(item)
```

index()

Encontra índice

```
lista.index(item)
```

Funções e Operadores

len()

Retorna tamanho

```
len(lista)
```

in

Verifica existência

```
item in lista
```

copy()

Cria cópia

```
lista.copy()
```

